Flaskify 🚀

A minimalist, high-performance Flask REST API Template Generator with built-in rate limiting, ML model support, and best practices. This is a great starting point for building robust APIs.

Features

- ⚡ High-performance REST API setup
- 🔒 Built-in rate limiting (configurable) with thread-safe implementation
- 🌐 CORS enabled (for cross-origin requests)
- 📁 Clear project structure
- 🔄 Version control ready (Git pre-initialized)
- 📦 Essential dependencies with fixed versions
- 🎯 Automatic environment detection and configuration
- 🔐 Automatic secure secret key generation
- ⚠️ Enhanced error handling and validation
- 🎨 Color-coded CLI output
- 💻 Cross-platform compatibility (Windows/Linux/Mac)
- 🔍 Project name validation
- 📊 Enhanced logging and debugging
- 🤖 ML model integration support
- 🧠 Built-in model training structure
- 📂 Organized model management

Installation

Linux/Mac

```
# Install Flaskify CLI
curl -s https://raw.githubusercontent.com/DonArtkins/flaskify/master/flaskify-install.sh | bash
```

Windows

```
# Install Flaskify CLI
iex ((New-Object System.Net.WebClient).DownloadString('https://raw.githubusercontent.com/DonArtkins
```

Quick Start

1. **Create a New Project:**

```
flaskify create my-api
cd my-api
```

2. **Activate the Virtual Environment:**

```
# Linux/Mac
source venv/bin/activate

# Windows
.\venv\Scripts\activate
```

3. **Run the API:**

```
python run.py
```

The API will start with the configured environment settings. You'll see color-coded output in your terminal.

API Structure

The project follows a clear structure:

```
my-api/
├── app/
│   ├── api/
│   │   └── v1/              # API version 1
│   │       ├── __init__.py  # Initializes the v1 API
│   │       └── routes.py    # Defines API endpoints
│   ├── config/
│   │   └── config.py        # Enhanced configuration with environment support
│   ├── models/             # Store your ML models here
│   │   ├── __init__.py
│   │   ├── training/      # Training scripts
│   │   └── trained_models/ # Saved model weights
│   ├── services/          # Business logic and model inference
│   │   └── __init__.py
│   ├── utils/
│   │   └── helpers.py       # Thread-safe rate limiting and utilities
│   └── __init__.py          # Application factory with enhanced error handling
├── tests/                   # Add your unit tests here
├── docs/                    # API documentation
├── venv/                    # Virtual environment
├── .env                     # Environment variables with auto-generated secure keys
├── .gitignore              # Enhanced Git ignore patterns
├── LICENSE                  # License information
├── CONTRIBUTING.md         # Contribution guidelines
├── README.md              # Project documentation
├── requirements.txt       # Pinned Python package dependencies
```

```
└── run.py                # Enhanced application entry point with environment support
```

Configuration

API configuration now supports multiple environments (development, testing, production) through environment variables. The `.env` file is automatically created with secure defaults:

```
# Flask Configuration
FLASK_APP=run.py
FLASK_ENV=development
FLASK_DEBUG=True
FLASK_HOST=0.0.0.0
FLASK_PORT=5000

# API Configuration
API_TITLE=My Awesome API
API_VERSION=v1
RATE_LIMIT=1000
RATE_LIMIT_PERIOD=15

# Model Configuration
MODEL_PATH=app/models/trained_models

# Security
SECRET_KEY=<auto-generated-secure-key>
```

Important:

- A secure `SECRET_KEY` is automatically generated during project creation.
- Environment-specific configurations are now supported through the `config` classes.
- Enhanced security with proper environment variable handling.
- Configurable host and port settings.
- Model path configuration for ML integration.

ML Model Integration

The template now includes full support for ML model integration:

- 📦 Dedicated model storage structure
- 🎯 Training script templates
- 🔄 Model versioning support
- 🎯 Inference service structure
- 📊 Example model endpoints

Deployment

Docker

Docker

1. **Build the Image:**

```
docker build -t my-api .
```

2. **Run the Container:**

```
docker run -p 5000:5000 \
  -e SECRET_KEY=$(python3 -c "import secrets; print(secrets.token_hex(32))") \
  my-api
```

Heroku

```
heroku create my-api
heroku config:set SECRET_KEY=$(python3 -c "import secrets; print(secrets.token_hex(32))")
git push heroku main
```

Railway

```
railway init
railway up
```

Rate Limiting

The API includes an enhanced thread-safe rate limiting implementation:

- **Default:** 1000 requests per 15 minutes
- **Thread-safe:** Properly handles concurrent requests
- **Configuration:** Adjust through environment variables
- **Enhanced error responses:** Includes retry-after headers
- **IP-based tracking:** Properly tracks request limits per client

Best Practices

Flaskify now includes additional best practices:

- ✅ **RESTful Routing:** Follow RESTful principles for API design
- ✅ **Thread-safe Rate Limiting:** Protection against concurrent request abuse
- ✅ **Enhanced Error Handling:** Comprehensive error handling with proper status codes
- ✅ **Environment Management:** Support for development, testing, and production environments
- ✅ **Secure Defaults:** Auto-generated secure keys and safe defaults
- ✅ **Cross-Platform Support:** Works reliably across Windows, Linux, and Mac

- ✅ **Input Validation:** Project name and configuration validation
- ✅ **Enhanced Logging:** Better debugging and monitoring support
- ✅ **CORS Support:** Configurable Cross-Origin Resource Sharing
- ✅ **Clean Project Structure:** Well-organized and maintainable codebase
- ✅ **ML Integration:** Built-in support for machine learning models
- ✅ **Model Management:** Structured approach to model versioning and deployment
- ✅ **Service Layer:** Clean separation of business logic and API endpoints

Contributing

See CONTRIBUTING for details on how to contribute to this project.

License

MIT License. See LICENSE for more information.